前端技术文档提案

1. 系统架构概述与分层定义

为满足系统的高内聚低耦合设计要求，并兼顾演示环境（单机部署）与生产环境（分布式部署）的兼容性，本项目对“前端系统”进行了逻辑分层与任务拆解。我们将前端业务划分为“用户表现层（前端客户服务程序）”与“渲染控制中台（UE绘制程序 + 控制与数据中台）”两大部分。

尽管在演示阶段各模块可能部署于同一台物理主机，但各组件之间严格遵循网络通信协议进行交互，确保架构具备物理分离的能力：

- **前端客户服务程序**：负责用户交互与轻量级数据可视化，可运行于任意支持 Web 标准的终端。
- **控制与数据中台**：作为前端系统的逻辑核心，负责信令交换、数据分发与持久化存储。
- **UE 画面绘制程序**：作为计算密集型节点，负责三维场景渲染与视频流编码。

各模块间通过 HTTP、**Socket.IO** 及 WebRTC 协议建立连接，实现逻辑上的完全解耦。

---

2. 详细模块设计与技术规范

2.1 前端客户服务程序 (Client Service Application)

- **技术栈（建议）**：Vue.js / React, ECharts, Pixel Streaming SDK, Axios
- **功能描述与工作内容**：
  - **交互式回放控制台**：这是本系统的核心界面。除了常规的视频流窗口外，需开发一个**高精度的“仿真时间轴（Timeline）”**。包含播放、暂停、倍速播放、拖拽跳转等功能。前端根据进度条的时间戳，向中台请求对应时刻的场景状态数据。
  - **场景漫游交互层**：虽然画面是历史数据的回放，但用户的观察视角是实时的。需捕获鼠标与键盘事件（WASD/旋转），实时发送给 UE 渲染端。**通信方式由原定的 WebSocket 调整为 Socket.IO 事件（Event-Based），以确保指令的可靠送达与自动重连。**
  - **仿真配置与任务提交**：设计参数配置面板，支持将复杂的灾害参数打包发送给中台。提交后，前端进入“计算等待”状态或“进度监视”状态，待后端计算完成后解锁回放功能。
  - **多维数据可视化**：与时间轴联动，当进度条拖动到 T 时刻，侧边栏的 ECharts 图表（如毒气浓度曲线、人群密度分布）也应同步定位到 T 时刻的数据点，实现 3D 画面与 2D 数据的联动展示。

2.2 控制与数据中台 (Control & Data Middleware)

- **技术栈（建议）**：Node.js (Express/Koa), MySQL/MongoDB, **Socket.IO Server**
- **功能描述与工作内容**：
  - **仿真数据缓冲与序列化**：这是架构转型的关键。中台不再单纯做实时的“二传手”，而是作为**“数据仓库管理者”**。在后端进行快速仿真时，中台负责接收全量数据帧，按 `Simulation_ID` 和 `Time_Step` 建立索引并存入数据库。
  - **按需数据分发 (On-Demand Streaming)**：响应前端的播放请求。当前端进度条位于 T 秒时，中台从数据库（或内存缓存）中精准提取 T 秒的数据帧，通过 **Socket.IO 向 UE 渲染端发射（Emit）数据事件**。支持“连续流式发送”（播放模式）和“单帧发送”（暂停/跳转模式）。**相比原生 WebSocket，Socket.IO 提供了心跳检测与断线自动重连机制，更适合不稳定的演示网络环境。**

- **信令服务与连接管理**：维持前端与 UE4 之间的 WebRTC 信令握手，确保视频流通道的稳定性。

2.3 UE 画面绘制程序 (UE Visualization Renderer)

- **技术栈（建议）**：Unreal Engine 4.26, Pixel Streaming Plugin, **Socket.IO Client Plugin**, Blueprints / C++
- **功能描述与工作内容**：
  - **无状态渲染引擎**：UE 端设计为“无记忆”的渲染器。它不关心当前是在算第一秒还是最后一秒，它只负责**“将收到的那一帧数据可视化”**。这意味着 UE 可以瞬间从第 10 秒的状态跳变到第 500 秒的状态，而不依赖物理引擎的连续性。
  - **交互式摄像机漫游**：保留完整的 GameMode 和 Pawn 逻辑。即使场景中的水和人是根据历史数据静止或播放的，UE 里的摄像机依然响应用户的实时控制指令，提供沉浸式的观察体验。
  - **帧数据解析与可视化**： 开发蓝图逻辑，**绑定 Socket.IO 的监听事件（Bind Event to "UpdateFrame"）**。解析中台推送的 JSON 对象，将环境数据（水位高度、烟雾浓度/类型）、人群数据（人员坐标、朝向、状态）转化为 UE 场景中的视觉效果；
  - **实时视频推流**：持续将渲染画面编码推流给前端。

---

3. 流程演示 (Process Demonstration)

本系统采用“离线计算 - 实时回放”的工作流，具体步骤如下：

1. **参数配置与计算触发**：用户在前端设置灾害参数（如：500人，突发洪水），点击“开始仿真”。指令发送至中台，中台调度后端开始计算。

2. **后台高速仿真与存储**：

   - 后端算法模块以最高速度运行（不需等待渲染），快速生成从第 0 秒到结束时刻的所有人群与环境数据帧。
   - 中台接收这些数据，将其结构化存入数据库。
   - 注：此*时*前端可能*显*示*进*度条“仿真*计*算中... *80%*”，*暂*不*显*示 *3D* 画面。

3. **交互式回放与复盘**：

   - 计算完成后，前端加载视频流窗口和时间轴。
   - **默认播放**：中台按正常时间倍率，顺序读取数据库中的每一帧发给 UE，UE 渲染后将画面传回前端。
   - **手动跳转**：用户将进度条拖动到“05:30”时刻。中台立即检索该时刻的数据发给 UE。UE 瞬间刷新场景中所有人和水的位置。
   - **自由观察**：用户点击“暂停”，此时场景中的人和水静止在 05:30 的状态。用户使用键盘 WASD 控制摄像机，在静止的场景中自由飞行，近距离观察某个被水淹没的角落或人群的拥堵情况。

4. **大模型预案推演（可选分支）**：

   - 在回放过程中，用户可截取某一段数据发送给大模型请求预案。
   - 若应用预案，系统将以当前帧为起点，结合预案参数，再次触发后端进行新一轮的“接力仿真计算”，生成新的后半段数据供用户对比观看。

---

4. 前端与其他模块对接规范

(一) 与后端仿真引擎对接

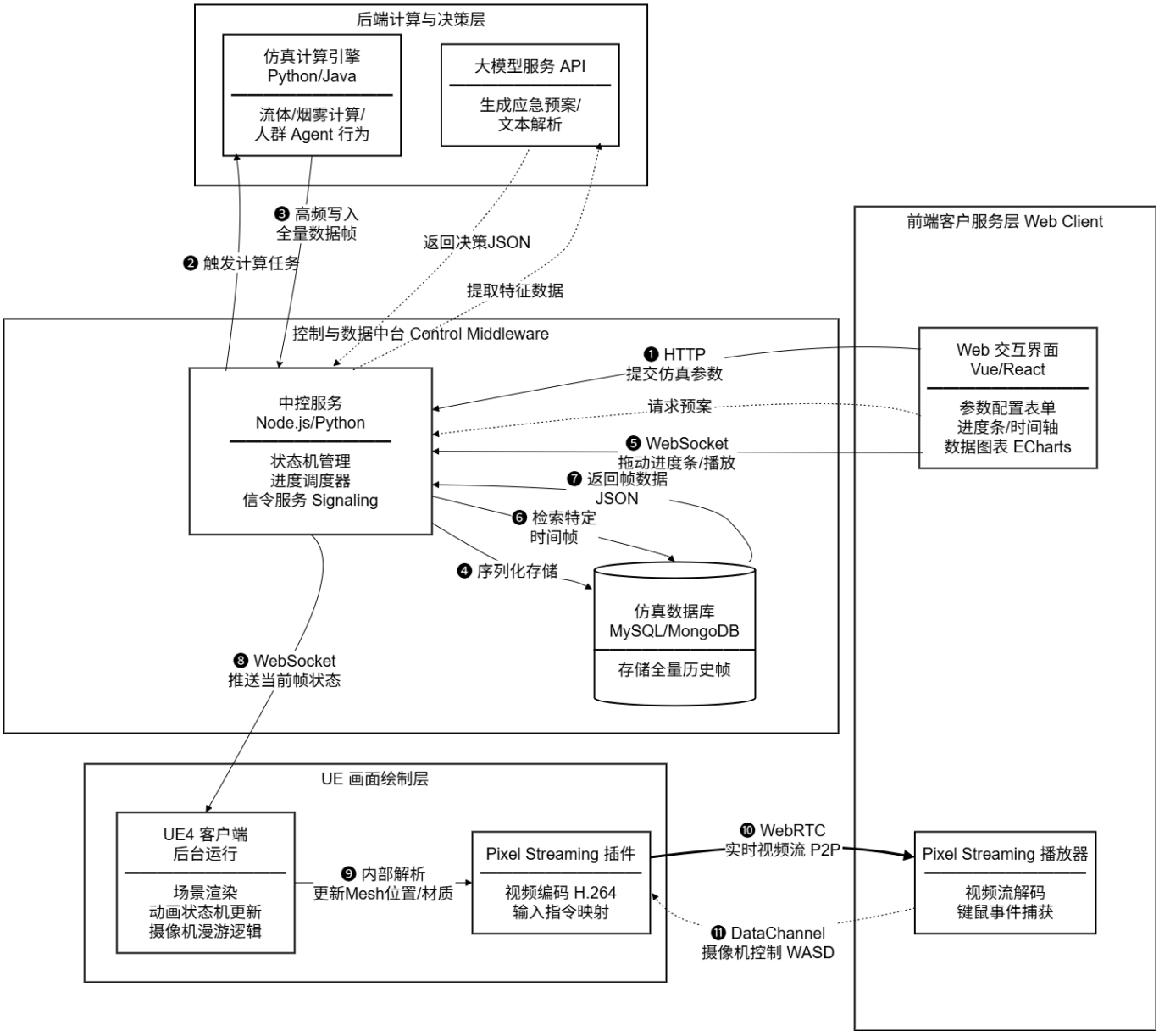
- **(1) 接口类型**：HTTP POST/GET + **Socket.IO**；
- **(2) 核心接口**：
  - 初始化配置：`POST /api/init` ... *(不变)*
  - **帧数据推送**：后端通过 **Socket.IO Event** 向前台控制与数据中台推送全量帧数据，避免 TCP 握手开销；
  - 计算进度查询：... *(不变)*

(二) 与大模型 API 对接 *(内容不变)*

(三) 前端内部模块对接

- **(1) 用户表现层 ↔ 控制与数据中台**：通过 HTTP（配置提交/状态查询）、**Socket.IO（帧数据请求/摄像机控制指令/播放进度同步）**；
- **(2) 控制与数据中台 ↔ UE 画面绘制程序**：通过 **Socket.IO（帧数据推送 Event / 摄像机控制响应 Event）**、WebRTC 信令（视频流通道）；
- **(3) 数据格式**：所有 JSON 数据采用 UTF-8 编码，字段名使用小驼峰命名。**Socket.IO 事件名约定如下**：
  - `UpdateFrame`: 中台推给 UE 的场景数据帧。
  - `ControlCamera`: 前端推给 UE 的漫游指令。
  - `SimState`: 仿真状态同步（播放/暂停/结束）。

架构图


后端计算与决策层
仿真计算引擎
Python/Java
流体/烟雾计算/
人群 Agent 行为
大模型服务 API
生成应急预案/
文本解析
❸ 高频写入
全量数据帧
返回决策JSON
❷ 触发计算任务
提取特征数据
前端客户服务层 Web Client
控制与数据中台 Control Middleware
❶ HTTP
提交仿真参数
Web 交互界面
Vue/React
参数配置表单
进度条/时间轴
数据图表 ECharts
中控服务
Node.js/Python
状态机管理
进度调度器
信令服务 Signaling
请求预案
❺ WebSocket
拖动进度条/播放
❼ 返回帧数据
JSON
❻ 检索特定
时间帧
❹ 序列化存储
仿真数据库
MySQL/MongoDB
存储全量历史帧
❽ WebSocket
推送当前帧状态
UE 画面绘制层
UE4 客户端
后台运行
场景渲染
动画状态机更新
摄像机漫游逻辑
❾ 内部解析
更新Mesh位置/材质
Pixel Streaming 插件
视频编码 H.264
输入指令映射
❿ WebRTC
实时视频流 P2P
Pixel Streaming 播放器
视频流解码
键鼠事件捕获
⓫ DataChannel
摄像机控制 WASD